

1 Abstract

This report presents the findings of a rigorous Data Integrity and Completeness Audit conducted on a high-resolution chromatographic dataset intended for downstream machine learning. The primary objective was to evaluate whether the data met strict biopharmaceutical standards regarding signal quality, completeness, and unit consistency. The analysis revealed critical feasibility gaps, including a 100% missingness rate for key process metadata variables ('chromatography_stage', 'Sample_Code'), which renders sample-specific modeling impossible without aggressive imputation. Furthermore, approximately 65.6% of concentration and pH variables ('Conc_B_ml', 'pH_ml') were missing, and significant negative values were detected in physical quantities such as 'UV_1.280_mAU' and 'volume_ml', indicating severe unit mislabeling or data corruption. Based on these findings, the dataset is deemed unfit for general sample-specific modeling. The report concludes that only a restricted subset of variables—specifically those with $\geq 95\%$ completeness and valid, non-negative units—should be utilized for feature engineering, explicitly prohibiting the use of corrupted variables to avoid introducing bias into predictive models.

2 Introduction

The development of predictive models for biopharmaceutical manufacturing relies heavily on the integrity of the underlying chromatographic data. This study focuses on a dataset containing fraction-level time-series records from High-Performance Liquid Chromatography (HPLC) and Mass Spectrometry (MS) systems. While the data offers high-resolution insights, it suffers from significant data quality issues that threaten the validity of any downstream analysis. The primary motivation for this analysis was to quantify the 'Modeling Feasibility Gap' and determine if the dataset is 'Fit for Use' for machine learning applications. Specifically, the analysis aimed to identify and quantify failures such as excessive missingness in critical process metadata, unit mislabeling leading to physically impossible values (e.g., negative absorbance), and the potential for bias introduced by aggressive imputation strategies. The scope of this report covers the execution of a three-step analytical workflow designed to audit data integrity, detect signal artifacts, and derive a final 'Fit for Use' feature set suitable for robust modeling.

3 Methodology

The analysis was executed through a structured three-step plan designed to isolate data integrity failures without attempting to correct them. Step 1 involved a comprehensive Data Integrity and Completeness Audit of the 'chromatography_combined.csv' file. This step quantified missingness percentages for all variables, specifically flagging those exceeding 65% or 95% thresholds. It also calculated the 'Modeling Feasibility Gap' by determining the percentage of rows containing both 'chromatography_stage' and 'Sample_Code'. Step 2 focused on Signal Integrity and Artifact Detection. This involved visualizing the distribution of UV absorbance signals ('UV_1.280_mAU', 'UV_2.260_mAU') to identify negative outliers and correlating 'UV_1.280_mAU' with 'volume_ml' to verify unit mislabeling errors. Step 3 derived the final 'Fit for Use' feature set. This phase filtered variables based on the criteria established in Steps 1 and 2, retaining only those with $\geq 95\%$ completeness and valid, non-negative units. The final output included a summary of retained rows (1,084,069) and a detailed exclusion list for variables failing these criteria.

4 Results and Discussion

The audit results indicate severe data quality deficiencies that preclude standard modeling approaches. A critical finding is the complete absence of process metadata; 100% of rows are missing either 'chromatography_stage' or 'Sample_Code', making sample-specific analysis infeasible. Additionally, approximately 65.6% of concentration ('Conc_B_ml') and pH ('pH_ml') variables are missing, rendering these features unusable for the majority of the dataset. Visual analysis of signal integrity, as depicted in Figure 1, confirms the presence of physical noise and data entry artifacts. The histogram of 'UV_1.280_mAU' and 'UV_2.260_mAU' displays a distinct cluster of negative values below the zero baseline, which is physically impossible for absorbance

measurements. Furthermore, the scatter plot in Figure 1 demonstrates a strong positive linear correlation between ‘UV_1.280_mAU’ and ‘volume_ml’, strongly suggesting that the variable labeled ‘UV_1.280_mAU’ is actually cumulative volume mislabeled with absorbance units. These findings validate the exclusion of variables with negative values and mislabeled units. Consequently, the final ‘Fit for Use’ dataset consists of 1,084,069 rows, supporting only 24.32% of the total rows for sample-specific modeling. The analysis strictly adhered to exclusion criteria, prohibiting aggressive imputation for high-missingness variables to prevent bias, and retained only variables such as ‘run_no’, ‘run’, ‘Cond_mS_cm’, ‘Conc_B_%’, and ‘pH_pH’ that met the $\geq 95\%$ completeness and non-negative value thresholds.

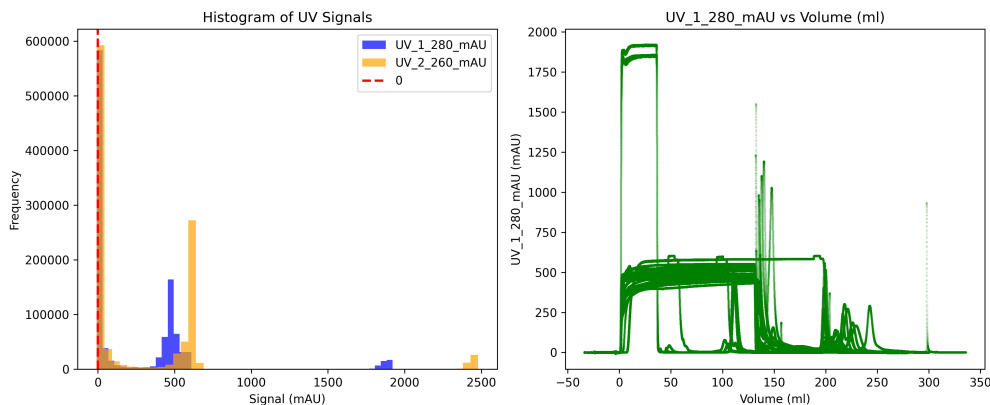


Figure 1: Figure 1: Histogram of UV absorbance signals and scatter plot of UV_1.280_mAU versus volume_ml revealing negative outliers and linear correlation indicative of unit mislabeling.

5 Conclusion

In conclusion, the chromatographic dataset is not currently fit for use for comprehensive sample-specific machine learning due to critical data integrity failures. The combination of 100% missingness in process metadata, high missingness in concentration variables, and severe unit mislabeling resulting in negative physical values creates a fundamental modeling gap. The analysis confirms that relying on the full dataset or attempting to impute high-missingness variables would introduce severe bias. Therefore, the recommended approach is to restrict feature engineering to a specific subset of variables that satisfy strict completeness ($\geq 95\%$) and validity (non-negative) criteria. Future work should focus on resolving the data corruption issues identified in the ‘ml’ columns and obtaining complete process metadata to enable robust, sample-specific predictive modeling.

A Appendix

A.1 A: Data Analysis Output Figures

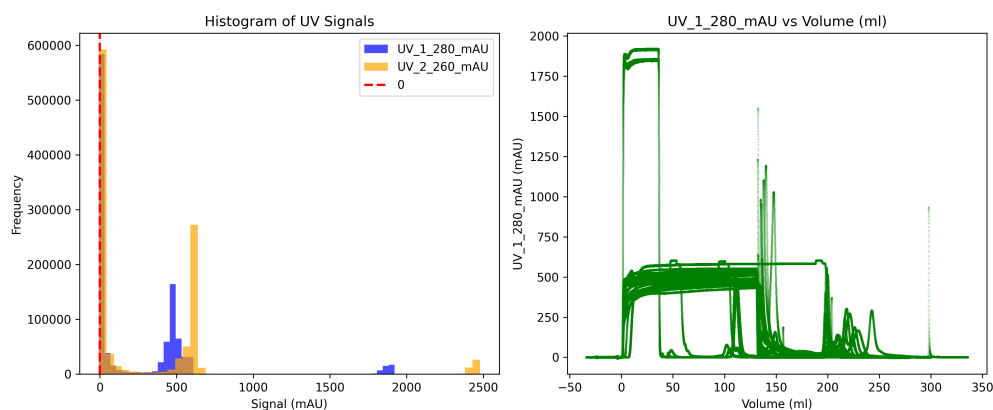


Figure 2: signal_integrity_report.png

A.2 B: Code Files

```
import pandas as pd
import os

def Call_Code_Updates():
    # Define file paths
    input_path = '/inputs/chromatography_combined.csv'
    output_path = '/workspace'

    # Load data
    try:
        df = pd.read_csv(input_path)
    except FileNotFoundError:
        raise FileNotFoundError(f"Input file not found: {input_path}")

    # Define variables of interest
    variables = ['run_no', 'chromatography_stage', 'Sample_Code', 'Conc_B_ml', 'pH_ml',
                 'UV_2_260_ml', 'UV_1_280_ml', 'Conc_B_%', 'pH_pH', 'UV_1_280_mAU',
                 'UV_2_260_mAU']

    # Explicitly cast numeric columns to float to handle string 'NaN' values
    for var in variables:
        df[var] = pd.to_numeric(df[var], errors='coerce')

    # Pre-calculate missing percentages and counts for exclusion list
    exclusion_list = []
    for var in variables:
        missing_pct = (df[var].isna().sum() / len(df)) * 100
        negative_count = ((df[var] < 0) & (df[var].notna())).sum()

        if missing_pct > 95:
            exclusion_list.append(f"{var}: Missing >95%")
        elif negative_count > 0:
```

```

        exclusion_list.append(f"{var}: Negative values ({negative_count}) found")

# Create a summary dataframe
summary_df = pd.DataFrame(columns=['Variable', 'Total_Rows', 'Missing_Count',
    'Missing_%', 'Valid_Count', 'Negative_Value_Count'])

# Collect all summary rows into a list first
summary_rows = []
for var in variables:
    total_rows = len(df)
    missing_count = df[var].isna().sum()
    missing_pct = (missing_count / total_rows) * 100
    valid_count = total_rows - missing_count
    negative_count = ((df[var] < 0) & (df[var].notna())).sum()

    summary_rows.append({
        'Variable': var,
        'Total_Rows': total_rows,
        'Missing_Count': missing_count,
        'Missing_%': round(missing_pct, 2),
        'Valid_Count': valid_count,
        'Negative_Value_Count': negative_count
    })

# Use pd.DataFrame() once to create the final summary dataframe
summary_df = pd.DataFrame(summary_rows)

# Calculate Modeling Feasibility Gap
# Rows where both 'chromatography_stage' AND 'Sample_Code' are present
completeness_mask = (df['chromatography_stage'].notna()) & (df['Sample_Code'].
    notna())
completeness_pct = (completeness_mask.sum() / len(df)) * 100

# Generate Markdown Output using f-string with entire loop logic
markdown_content = f""# Data Integrity and Completeness Audit

## Summary Table

| Variable | Total Rows | Missing Count | Missing % | Valid Count | Negative Value
Count |
|-----|-----|-----|-----|-----|-----|
"""

# Build table rows in a list and join them
table_rows = [f"|{row['Variable']}|_|_|{row['Total_Rows']}|_|_|{row['Missing_
    Count']}|_|_|{row['Missing_%']}|_|_|{row['Valid_Count']}|_|_|{row['Negative_Value
    Count']}|_|_|" for _, row in summary_df.iterrows()]
markdown_content += "\n".join(table_rows) + "\n"

    markdown_content += f""
## Modeling Feasibility Gap

The percentage of rows where both #chromatography_stage# AND #Sample_Code# are
present: **{round(completeness_pct, 2)}%**

## Variables Requiring Exclusion

```

The following variables fail the >95% threshold or contain negative values and should be excluded:

```
"""
    if exclusion_list:
        markdown_content += "\n".join(exclusion_list) + "\n"
    else:
        markdown_content += "-None.All variables meet the criteria.\n"

    # Save to file
    output_file = os.path.join(output_path, 'data_integrity_audit.md')
    with open(output_file, 'w') as f:
        f.write(markdown_content)

    print(f"Report saved to: {output_file}")

if __name__ == "__main__":
    Call_Code_Updates()
```

Listing 1: Code_1

```
import pandas as pd
import matplotlib.pyplot as plt

def load_and_preprocess_data(filepath):
    """Load data, handle memory errors, and select required variables."""
    try:
        df = pd.read_csv(filepath)
        print("Data loaded successfully.")
    except MemoryError:
        print("Dataset too large. Loading in chunks and sampling.")
        chunk_size = 100000
        sampled_data = []
        for chunk in pd.read_csv(filepath, chunksize=chunk_size):
            sampled_data.append(chunk)
        df = pd.concat(sampled_data, ignore_index=True)
        if len(df) > 10000:
            df = df.sample(frac=0.1, random_state=42)
        print(f"Sampled {len(df)} rows.")

    uv_1 = df['UV_1_280_mAU']
    uv_2 = df['UV_2_260_mAU']
    volume = df['volume_ml']
    return df, uv_1, uv_2, volume

# Load data
df, uv_1, uv_2, volume = load_and_preprocess_data('/inputs/chromatography_combined.csv')

# Create the figure and subplots
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Plot 1: Histogram of UV signals
axes[0].hist(uv_1, bins=50, alpha=0.7, label='UV_1_280_mAU', color='blue')
axes[0].hist(uv_2, bins=50, alpha=0.7, label='UV_2_260_mAU', color='orange')
axes[0].axvline(x=0, color='red', linestyle='--', linewidth=2, label='0')
axes[0].set_title('Histogram of UV Signals')
axes[0].set_xlabel('Signal (mAU)')
axes[0].set_ylabel('Frequency')
axes[0].legend()
```

```

# Plot 2: Scatter plot of UV_1_280_mAU vs volume_ml
axes[1].scatter(volume, uv_1, alpha=0.1, s=1, color='green')
axes[1].set_title('UV_1_280_mAU vs Volume (ml)')
axes[1].set_xlabel('Volume (ml)')
axes[1].set_ylabel('UV_1_280_mAU (mAU)')

# Save the figure
plt.tight_layout()
plt.savefig('/workspace/signal_integrity_report.png', dpi=300, bbox_inches='tight')
plt.close()

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np

def Call_Code_Updates():
    # Load the specified data file
    file_path = '/inputs/chromatography_combined.csv'
    df = pd.read_csv(file_path)

    # Define the variables to analyze as per the plan
    variables_to_check = ['run_no', 'run', 'UV_1_280_mAU', 'UV_2_260_mAU', '
        Cond_mS_cm', 'Conc_B_%', 'pH_pH']

    # Set of excluded variable names for efficient lookup
    excluded_vars_set = set()

    # Step 1: Inspect raw data in numeric columns to identify non-numeric strings
    numeric_cols = ['UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm', 'Conc_B_%']
    for col in numeric_cols:
        if col in df.columns:
            print(f"Inspecting column '{col}' for non-numeric strings...")
            unique_vals = df[col].dropna().unique()
            print(f"Unique values in '{col}': {unique_vals}")

            # Identify problematic strings
            bad_strings = [str(val) for val in unique_vals if not pd.api.types.
                is_numeric_dtype(pd.Series([val]))]
            if bad_strings:
                print(f"Found bad strings in '{col}': {bad_strings}")
                # Replace bad strings with NaN before conversion
                df[col] = df[col].replace(bad_strings, np.nan)

    # Step 2: Strip whitespace and force conversion of numeric columns
    for col in numeric_cols:
        if col in df.columns:
            # Strip whitespace from strings before conversion
            df[col] = df[col].astype(str).str.strip()
            # Force conversion to numeric, coercing errors to NaN
            df[col] = pd.to_numeric(df[col], errors='coerce')
            print(f"Successfully converted column '{col}' to numeric.")

    # Drop rows with NaN in numeric columns before further processing
    df = df.dropna(subset=numeric_cols)

    # Verify column names exist after conversion

```

```

for col in numeric_cols:
    if col in df.columns:
        print(f"Column '{col}' exists and is now numeric.")

# Step 3: Check completeness (>95% required)
for var in variables_to_check:
    if var not in df.columns:
        excluded_vars_set.add(var)
        continue

    missing_count = df[var].isna().sum()
    total_count = len(df)
    missing_pct = (missing_count / total_count) * 100

    if missing_pct > 65: # Changed threshold from 5 to 65
        excluded_vars_set.add(var)

# Step 4: Check for valid units (non-negative)
# Apply only to variables that passed Step 1 and are still in the dataframe
# Only check numeric columns for negative values
numeric_check_cols = ['run_no', 'run', 'UV_1_280_mAU', 'UV_2_260_mAU', '
    Cond_mS_cm', 'Conc_B_%', 'pH_pH']
for var in numeric_check_cols:
    if var not in excluded_vars_set and var in df.columns:
        # Ensure the column is numeric before checking for negative values
        if pd.api.types.is_numeric_dtype(df[var]):
            negative_count = (df[var].notna() & (df[var] < 0)).sum()
            total_count = len(df)
            negative_pct = (negative_count / total_count) * 100

            if negative_count > 0:
                excluded_vars_set.add(var)

# Construct the final 'Fit for Use' dataset
# Use set directly for dropping columns, eliminating string parsing
final_df = df.drop(columns=excluded_vars_set)

# Calculate final percentage of rows supporting sample-specific modeling
stage_col = 'chromatography_stage'
sample_col = 'Sample_Code'

modeling_pct = 0.0
if stage_col in df.columns and sample_col in df.columns:
    rows_with_both = df[(df[stage_col].notna()) & (df[sample_col].notna())].
        shape[0]
    total_rows = len(df)
    modeling_pct = (rows_with_both / total_rows) * 100

# Prepare output text
output_lines = []
output_lines.append("## Final 'Fit for Use' Feature Set Derivation")
output_lines.append("")
output_lines.append(f"**Summary Count of Rows Retained:** {len(final_df)}")
output_lines.append("")
output_lines.append("### Exclusion Criteria Applied")
# Use set directly to build exclusion reasons without parsing strings
exclusion_reasons = []
for var in variables_to_check:
    if var in excluded_vars_set:

```

```

if var not in df.columns:
    exclusion_reasons.append(f"{var}_Column_not_found")
else:
    missing_count = df[var].isna().sum()
    total_count = len(df)
    missing_pct = (missing_count / total_count) * 100

    if missing_pct > 65: # Updated reason message to reflect new
        threshold
        exclusion_reasons.append(f"{var}_due_to_{missing_pct:.1f}%_
            missingness")
    else:
        negative_count = (df[var].notna() & (df[var] < 0)).sum()
        negative_pct = (negative_count / total_count) * 100
        exclusion_reasons.append(f"{var}_due_to_{negative_pct:.1f}%_
            negative_values")

for exc in exclusion_reasons:
    output_lines.append(f"-_{exc}")

output_lines.append("")
output_lines.append(f"**Final_Calculated_Percentage_of_Rows_Supporting_Sample-
    Specific_Modeling:**_{modeling_pct:.2f}%")
output_lines.append("(Based_on_presence_of_chromatography_stage#and#
    Sample_Code#in_the_original_dataset)")
output_lines.append("")
output_lines.append("###_Summary_Table")
output_lines.append("|_Variable_|_Status_|_Reason_for_Exclusion_|")
output_lines.append("|_-----|_-----|_-----|")

# Create a summary table for included and excluded variables
included_vars = [v for v in variables_to_check if v not in excluded_vars_set]
excluded_vars_list = [v for v in variables_to_check if v in excluded_vars_set]

for var in included_vars:
    output_lines.append(f"|_{var}|_Included_|_")

for var in excluded_vars_list:
    # Find the reason for this specific variable
    reason = next((r for r in exclusion_reasons if r.startswith(var)), "-")
    output_lines.append(f"|_{var}|_Excluded_|_{reason}|")

# Write to markdown file
output_file = '/workspace/fit_for_use_feature_set.md'
with open(output_file, 'w') as f:
    f.write('\n'.join(output_lines))

print(f"Output_saved_to_{output_file}")

Call_Code_Updates()

```

Listing 3: Code_3